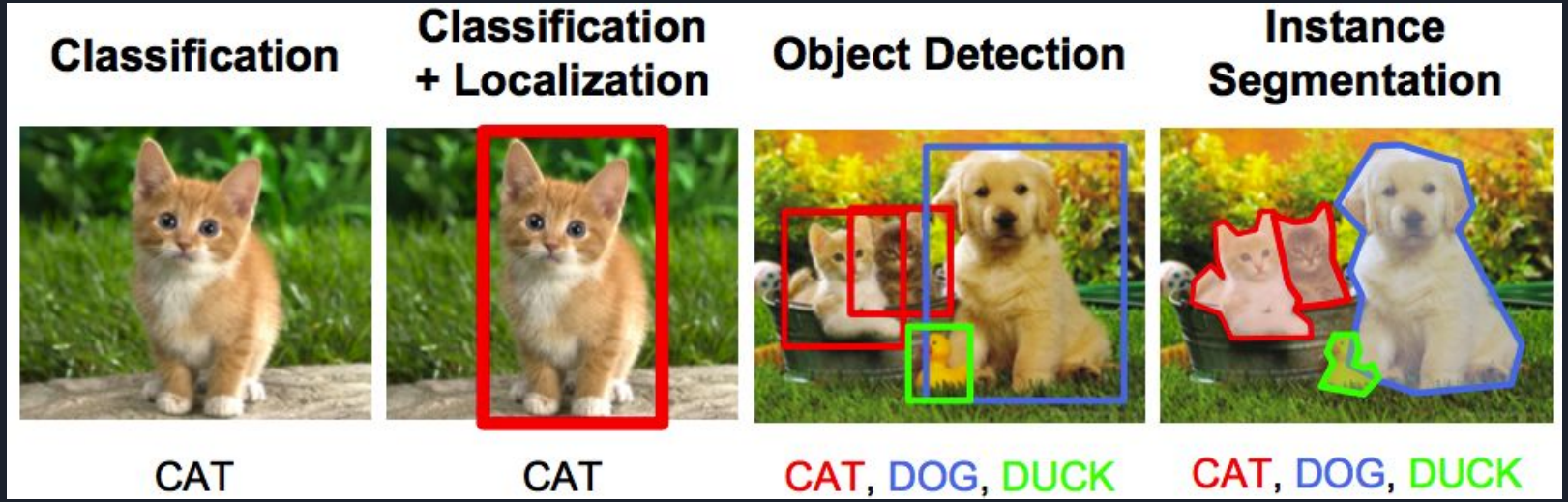
Object Detection

Engº Vinícius Araújo Santos

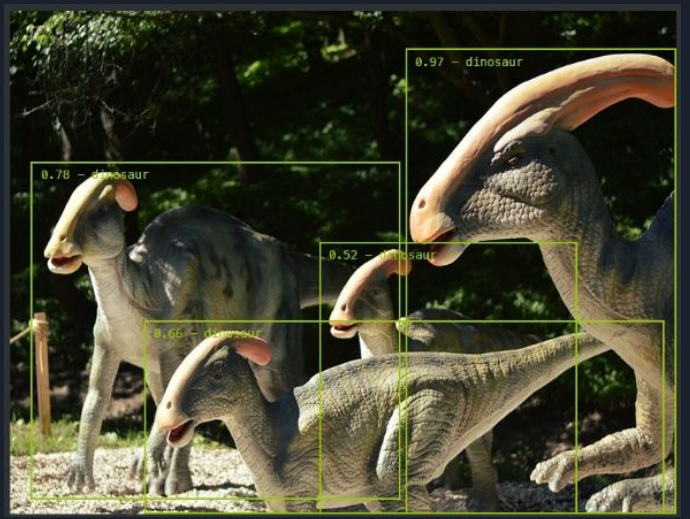


Object Detection

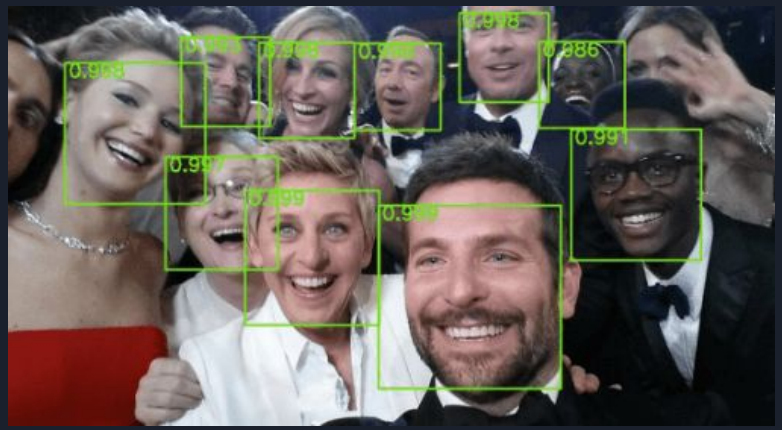


Simple Detection vs. Object Detection



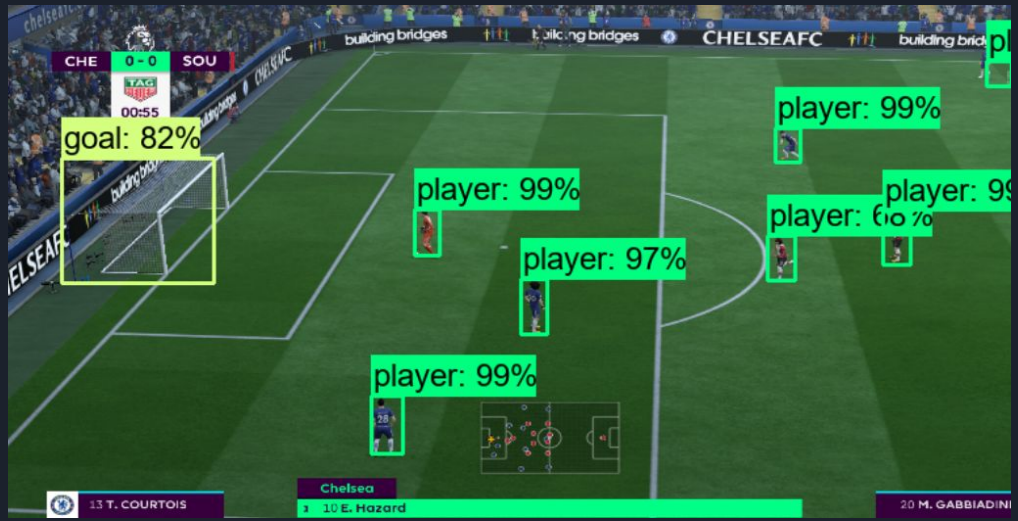


Aplicações de Object Detection

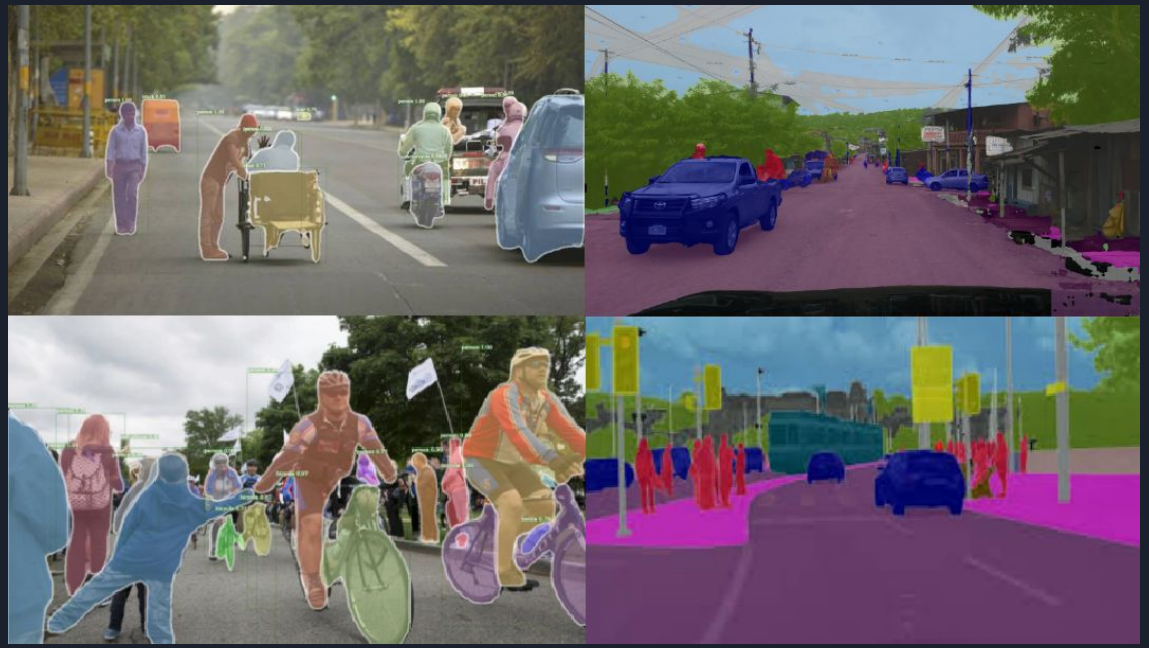




Aplicações de Object Detection



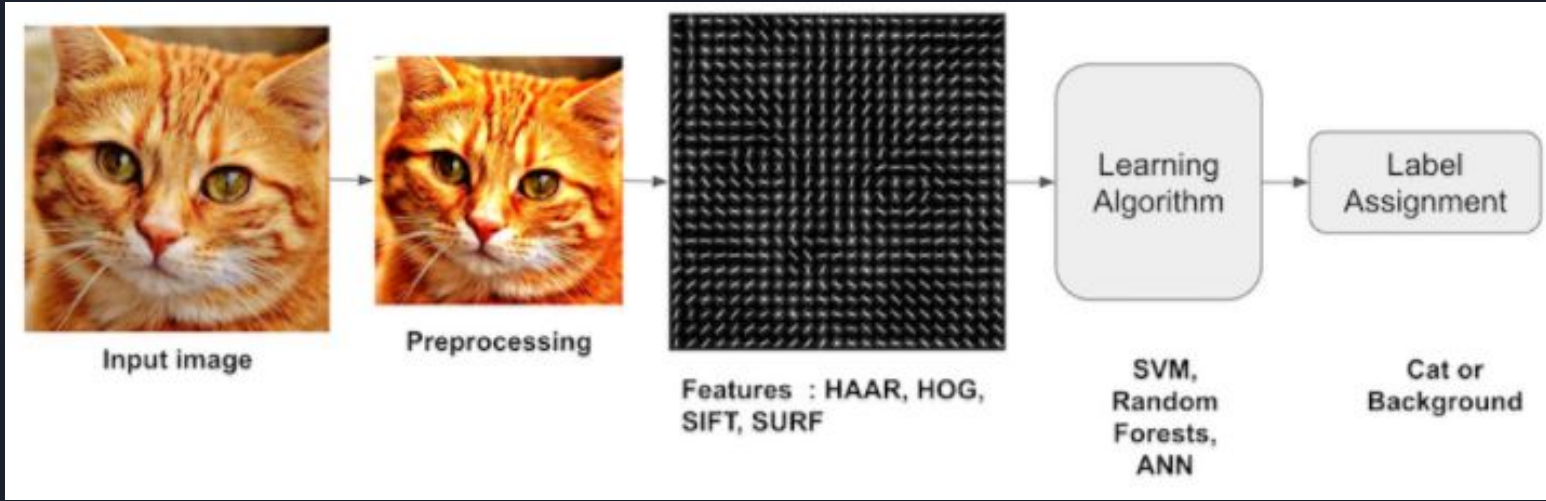
Object Segmentation



Desafios em Object Detection

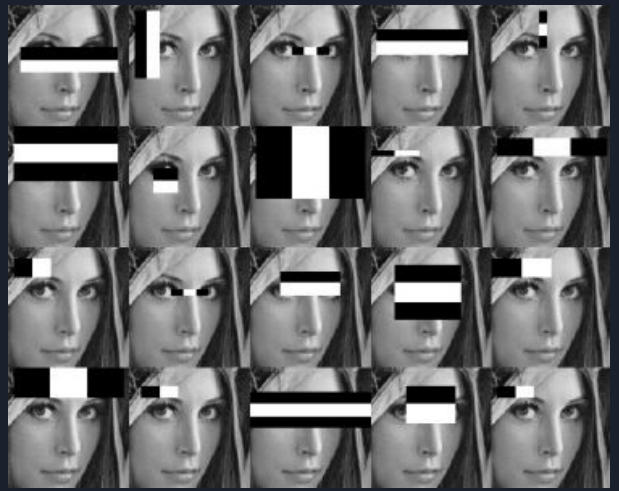
- Número variável de **objetos na imagem**.
- **Tamanho** variável dos objetos.
- **Modelagem**: solucionando 2 problemas de uma só vez
- **Posição** do objeto (bounding box) e **classificação** do objeto

Object Detection em Visão Computacional Clássica



Object Detection em Visão Computacional Clássica

- Modelo de **Viola-Jones**, proposto em 2001 por Paul Viola e Michael Jones:
  - Utiliza **janelamento deslizante** multi-dimensional.
  - Utiliza milhares de classificadores binários que utilizam **Haar features**.
  - Eficiente.
  - Simples implementação.



Object Detection em Visão Computacional Clássica



Object Detection em Visão Computacional Clássica

- **HOG** (Histogram of Oriented Gradients) features + SVM (support vector machine):
  - Também utiliza **janelamento deslizante** multi-dimensional.
  - **Melhor acurácia** que o método VIola-Jones para objetos em geral
  - Para **detecção de face** Viola-Jones é melhor
  - Mais **lento** em decorrência do cálculo de derivada para todos os pixels

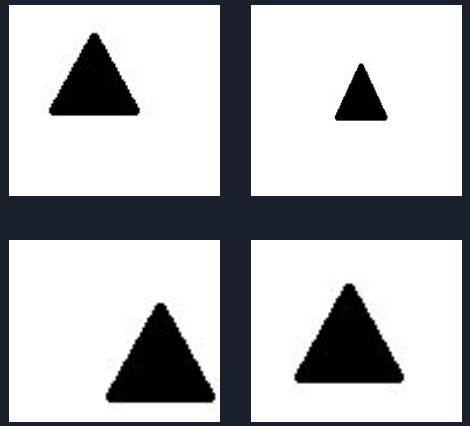


Object Detection e Deep Learning

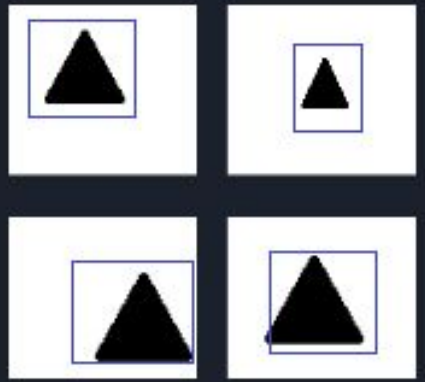
- Existem inúmeras arquiteturas dedicadas a esta tarefa
  - **YOLO** (You Only Look Once)
  - **SSD** (Single-Shot Detection)
  - **R-CNN** (Regions with Convolutional Network)
  - **Fast-RCNN**
  - **Faster-RCNN**
  - **R-FCNN** (Region-Based Convolutional Network)
  - **SqueezeDet**
  - Dentre outras...

Object Detection e Deep Learning

Funciona?



Funciona!

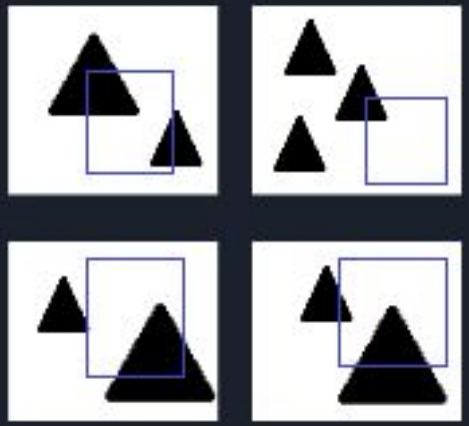


Object Detection e Deep Learning

Funciona?



NÃO Funciona!



Object Detection e Deep Learning

Por que não funciona?

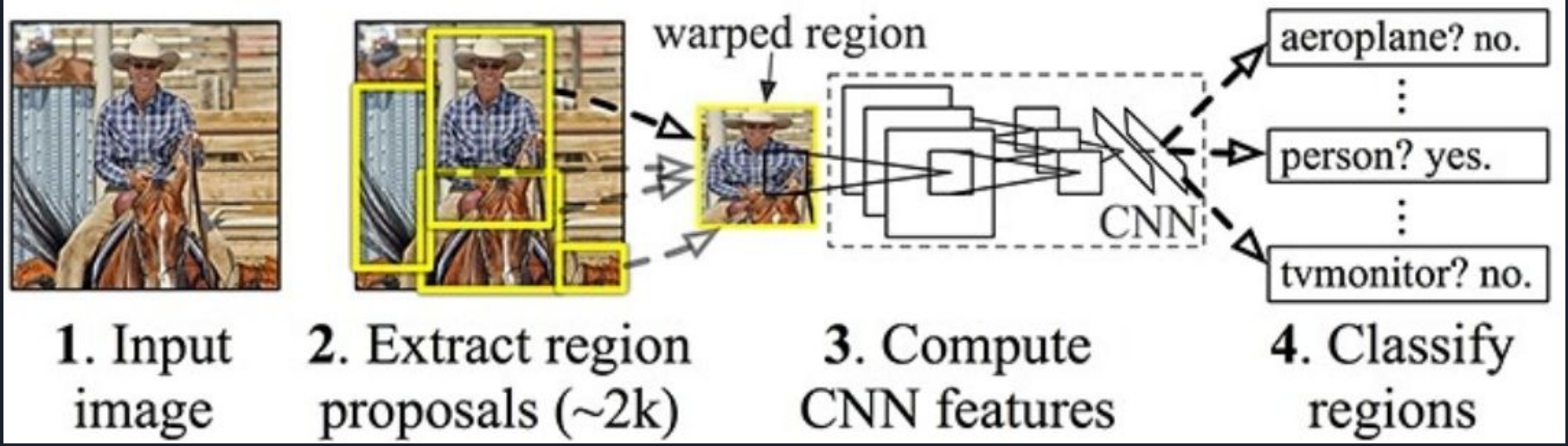
Como fazer funcionar?

Object Detection e Deep Learning





R-CNN - Regions with CNNs

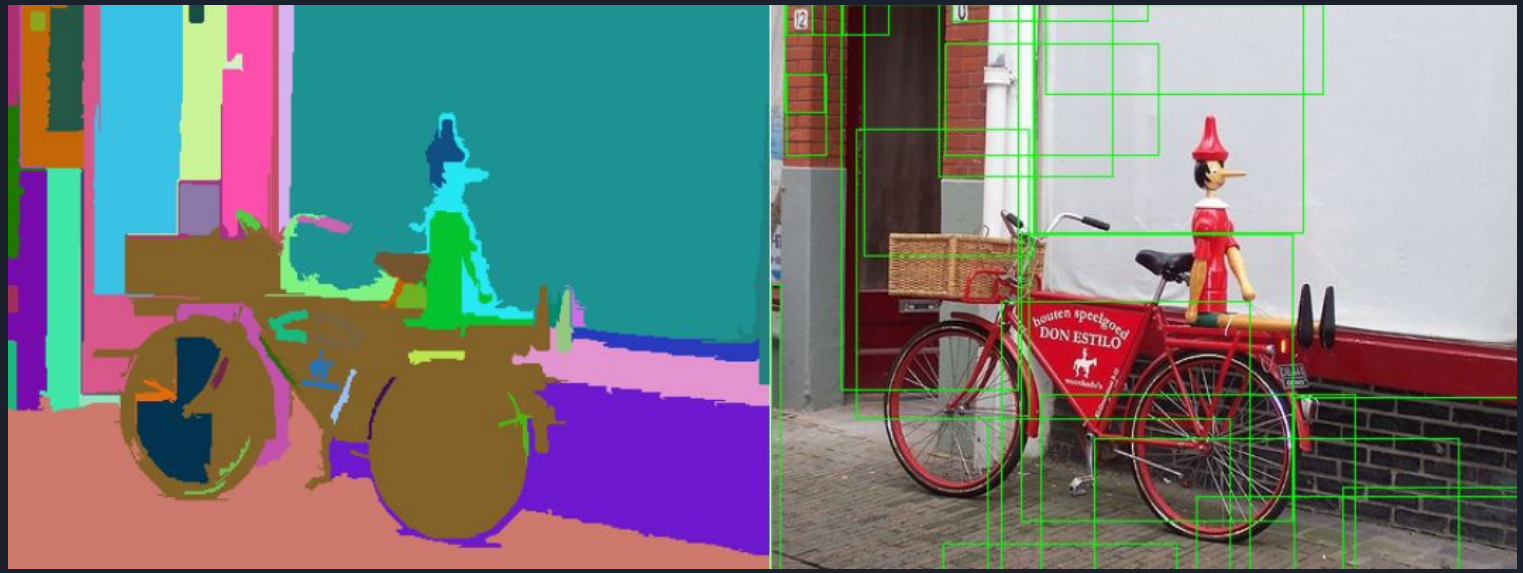


R-CNN

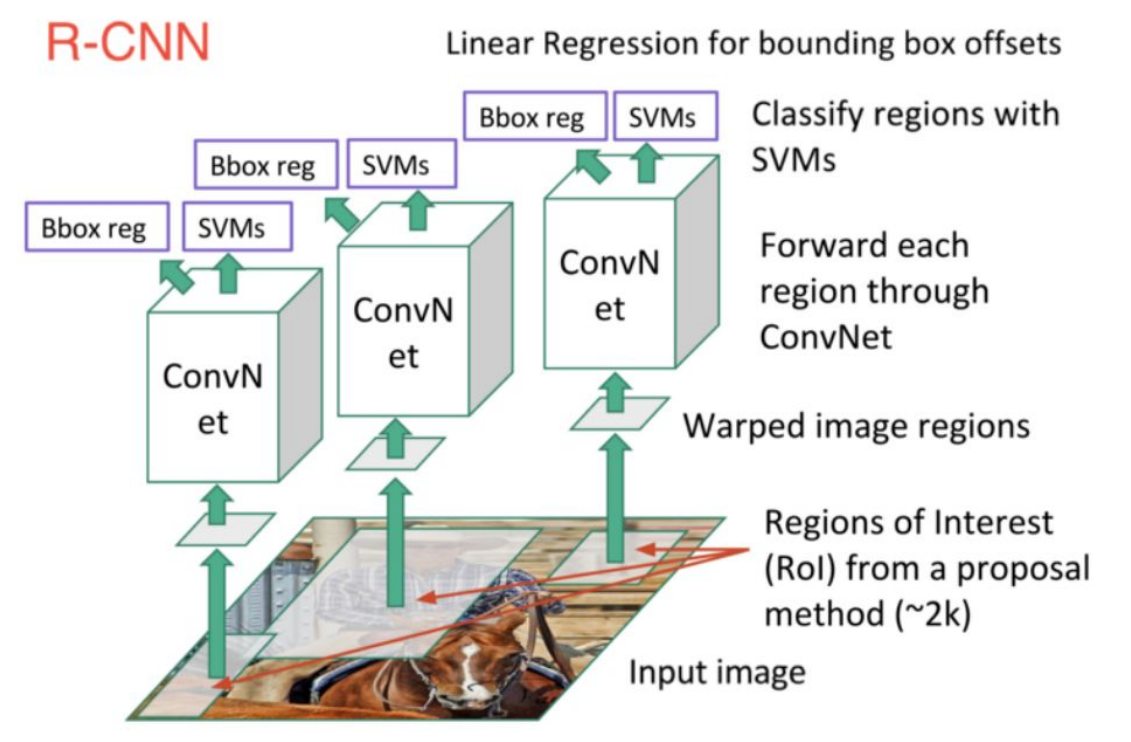
- Processo:
  - Extrai os possíveis objetos utilizando um método baseado em regiões (o mais popular é o **Selective Search**).
  - Extrai as características (features) das regiões selecionadas utilizando **CNN**.
  - Classifica cada região utilizando **SVM**.

- Problemas:
  - Processo de treinamento **lento** e **complexo**.

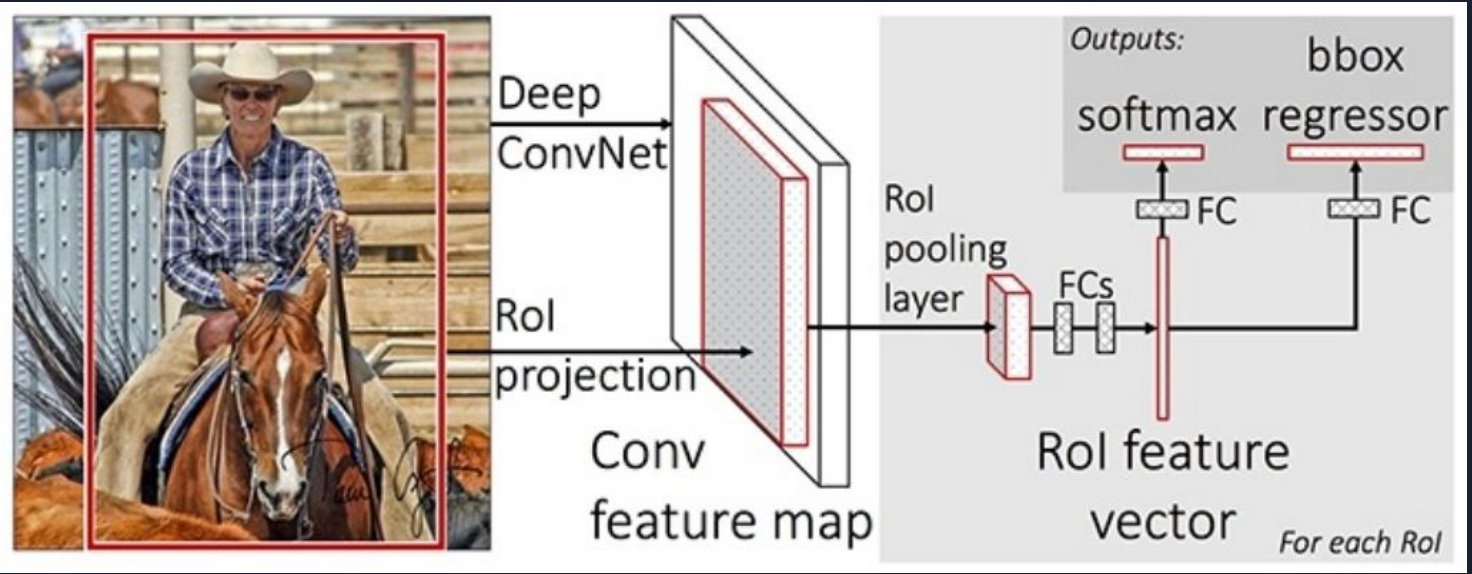
Selective Search



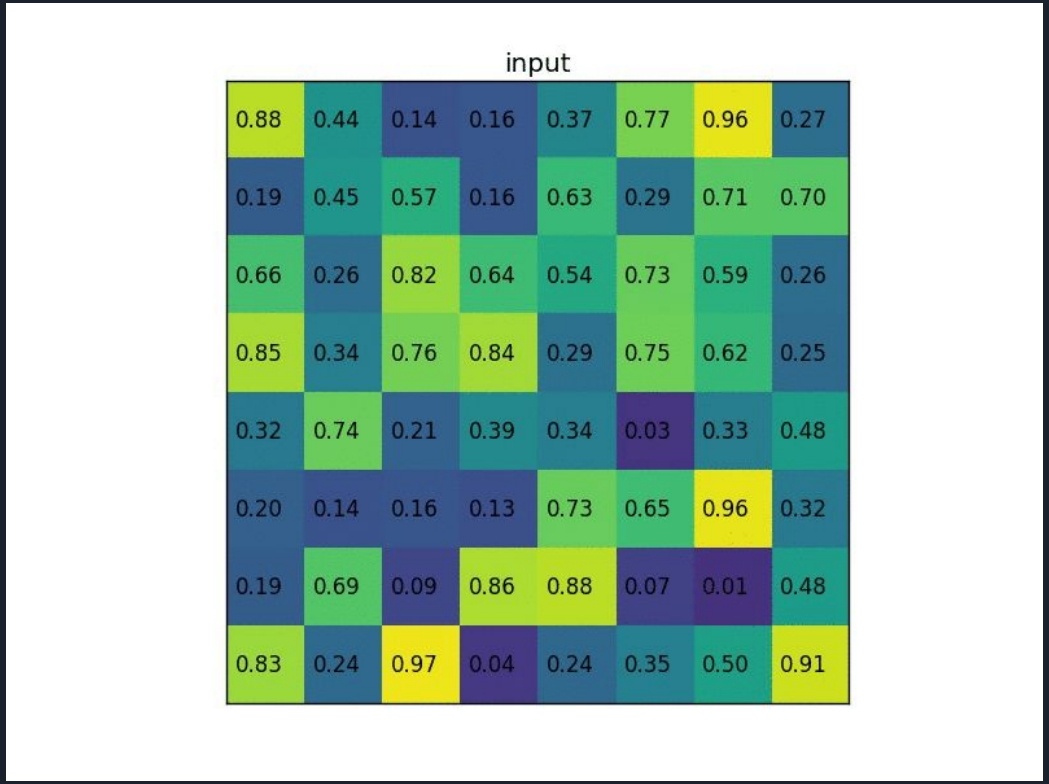
Selective Search na prática



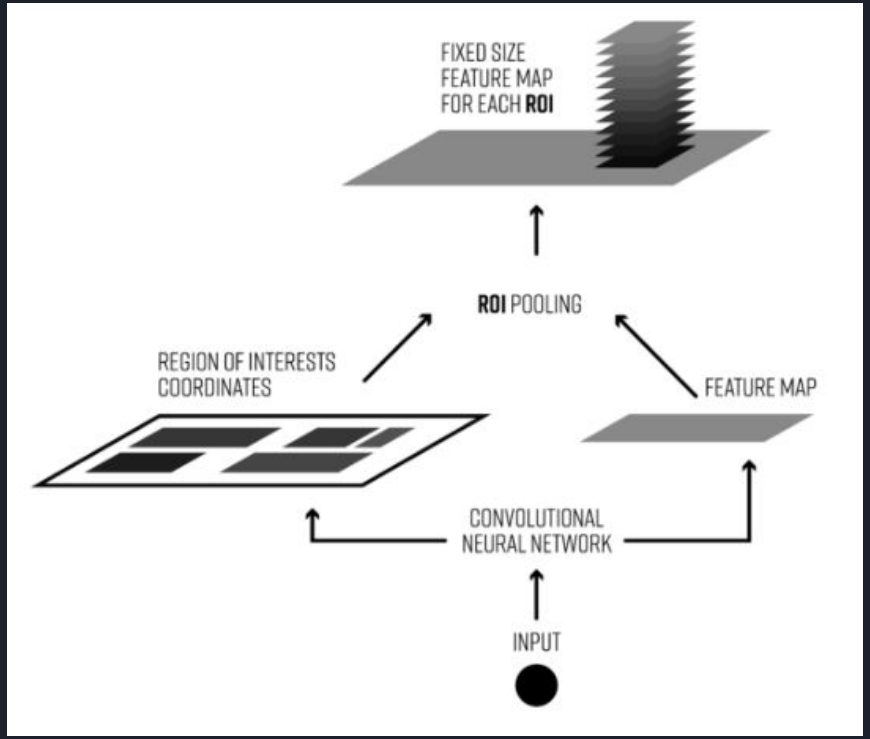
Fast R-CNN


Deep
ConvNet
RoI
projection
Conv
feature map
RoI
pooling
layer
FCs
Outputs:
softmax
bbox
regressor
FC
FC
RoI feature
vector
For each RoI

Fast R-CNN na prática



Region-of-interest (RoI) Pooling

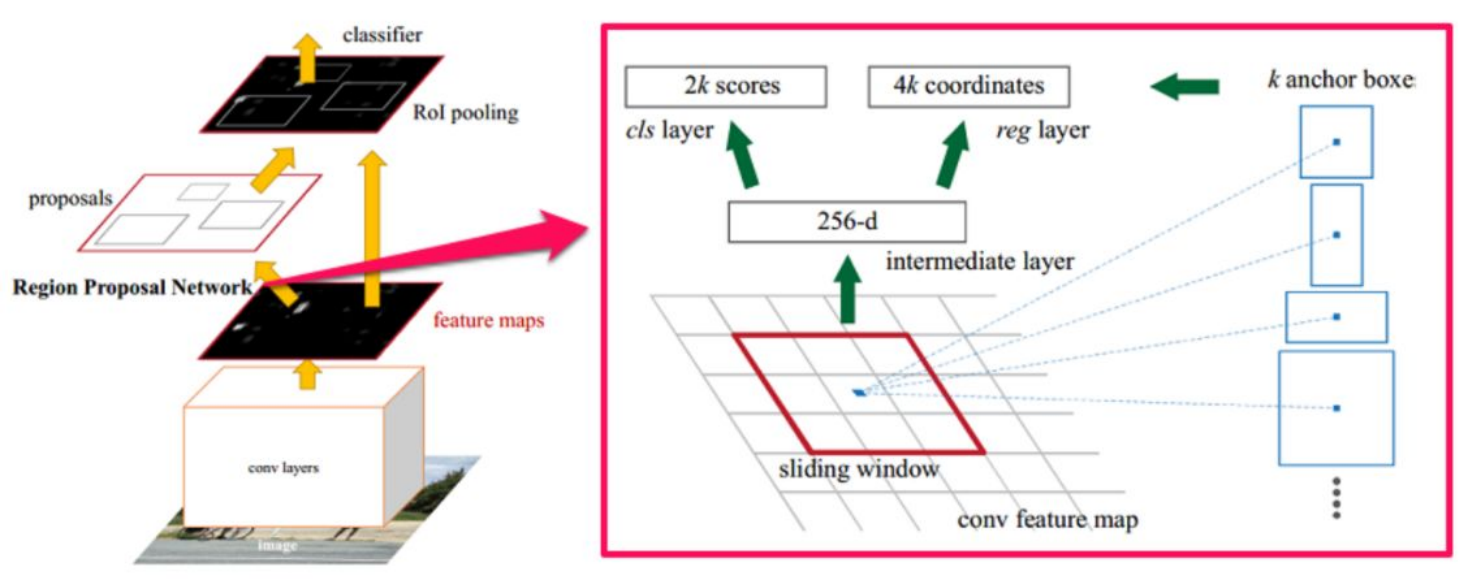


Fast R-CNN

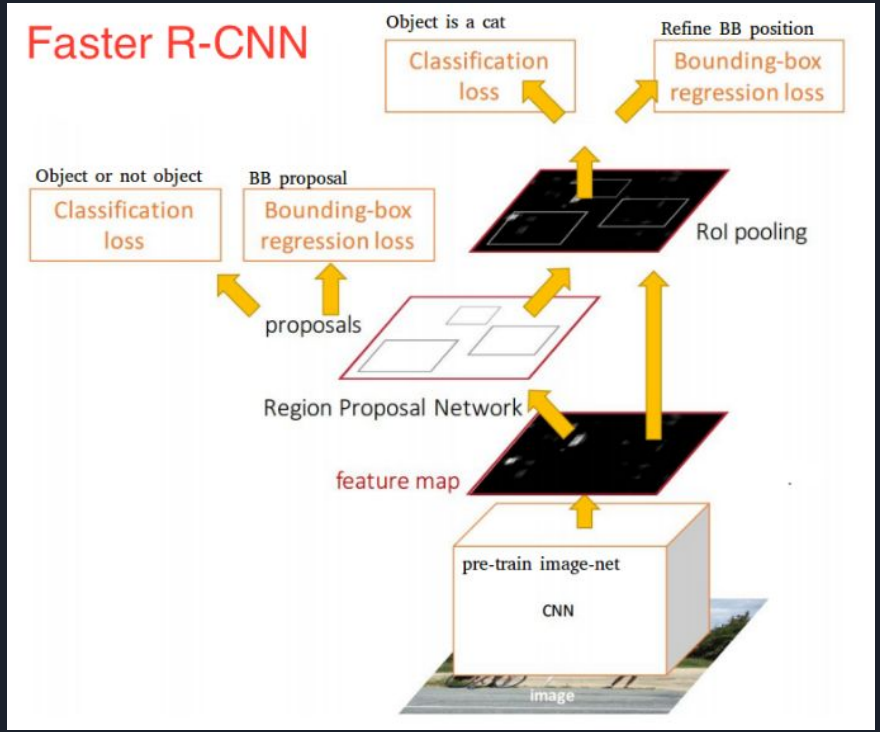
- Principal contribuição: **Region-of-Interest** (RoI) **Pooling**
  - Reuso do **mapa de características** extraída pela CNN.
  - Aumento considerável na **velocidade** de treinamento e testes.
  - Permite o treino de object detection **end-to-end** (substituindo SVM por FC layers).
  - Necessário um **algoritmo de seleção de regiões**.
  - Inferência pouco eficiente.

Faster R-CNN

- **RPN** + Fast R-CNN
- Deixar de propor **classificação aleatória** ou usar **Seletive search**



Region Proposal Network (RPN)



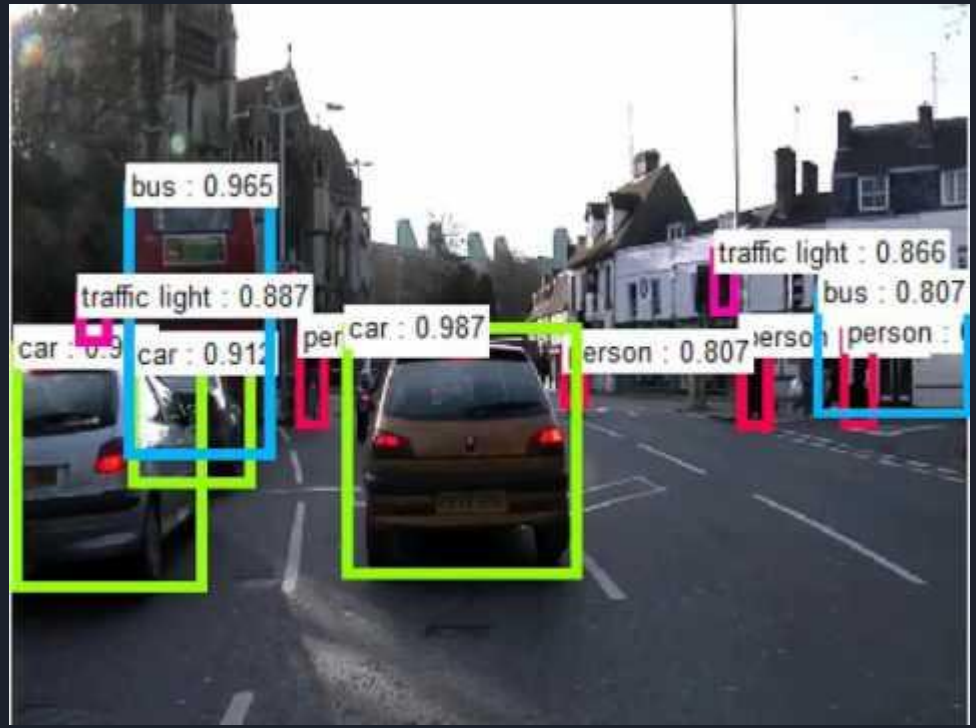
Faster R-CNN

- **Principal contribuição**: **Region Proposal Network** (RPN).
- **Melhor acurácia** em Detecção de Objetos
- Muito **mais rápida** que a Fast R-CNN

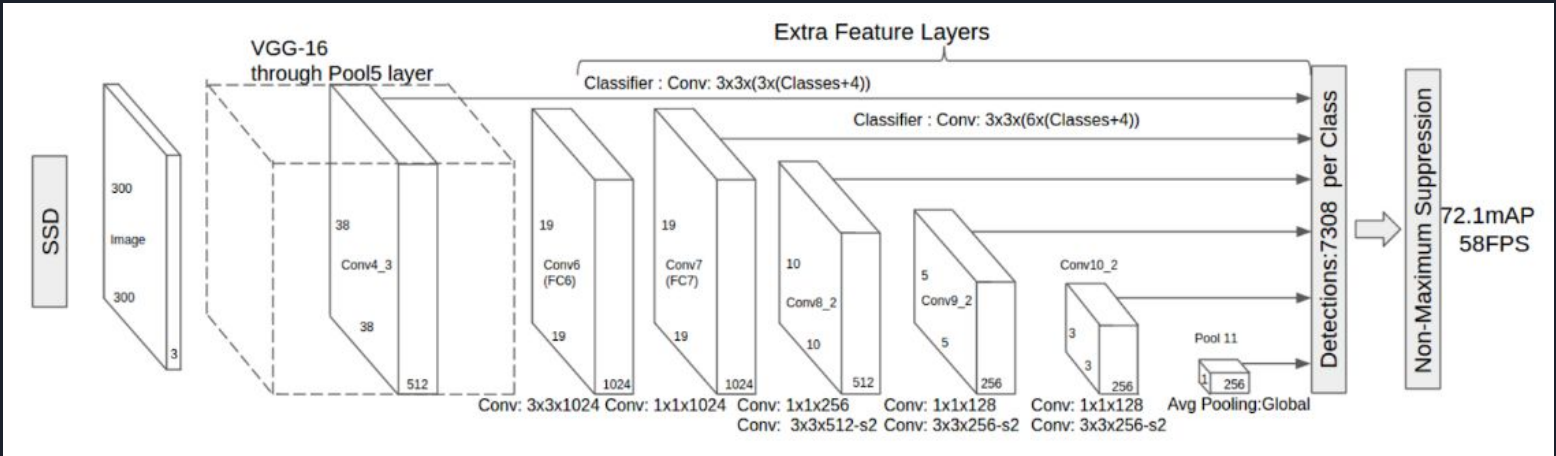
Nem tudo são flores!

- **Lenta** para aplicação em **tempo real**
- Rede de **referência** em object detection
- Inspirou vários outros modelos atuais

Faster R-CNN


bus : 0.965
traffic light : 0.887
car : 0.987
traffic light : 0.866
bus : 0.807
person :
erson : 0.807

SSD - Single Shot Multibox Detector



Funcionamento da SSD

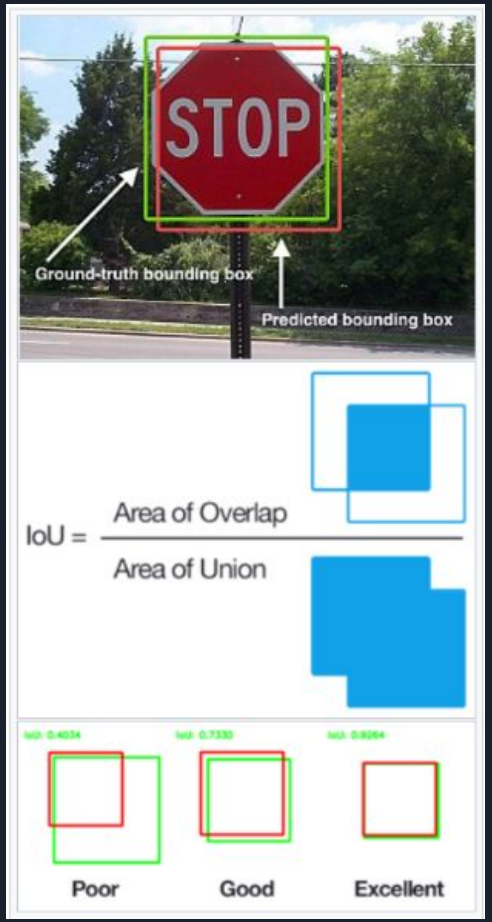
- Imagem passa por uma série de **camadas convolucionais** com diferentes filtros
- Para cada **feature map**, utilize um filtro convolucional **3x3** para avaliar um **conjunto menor** de bounding boxes.
- Equivalente às **anchor boxes** da **Faster R-CNN**.

Funcionamento da SSD

- Para cara **bounding box**, simultaneamente faça a predição de
  - **bounding box offset**
  - **probabilidade** da box pertencer a cada **classe**
- Durante o **treinamento**, compare a bounding box original com a prevista utilizando **IoU**.
- A bounding box com a "**melhor nota**" será marcada como "**positiva**", juntamente com todas aquelas com IoU com **confiança** maior o limiar de **0,5**.

IoU - Intersection over Union
Indice de Jaccard

- Método estatístico para comparação de **similaridade** de um conjunto de amostras.



Diferença entre SSD e anteriores (RCNNs)

- Nas **RCNNs** ao classificar temos a **mínima confiança** de existir algum **objeto** na região escolhida.

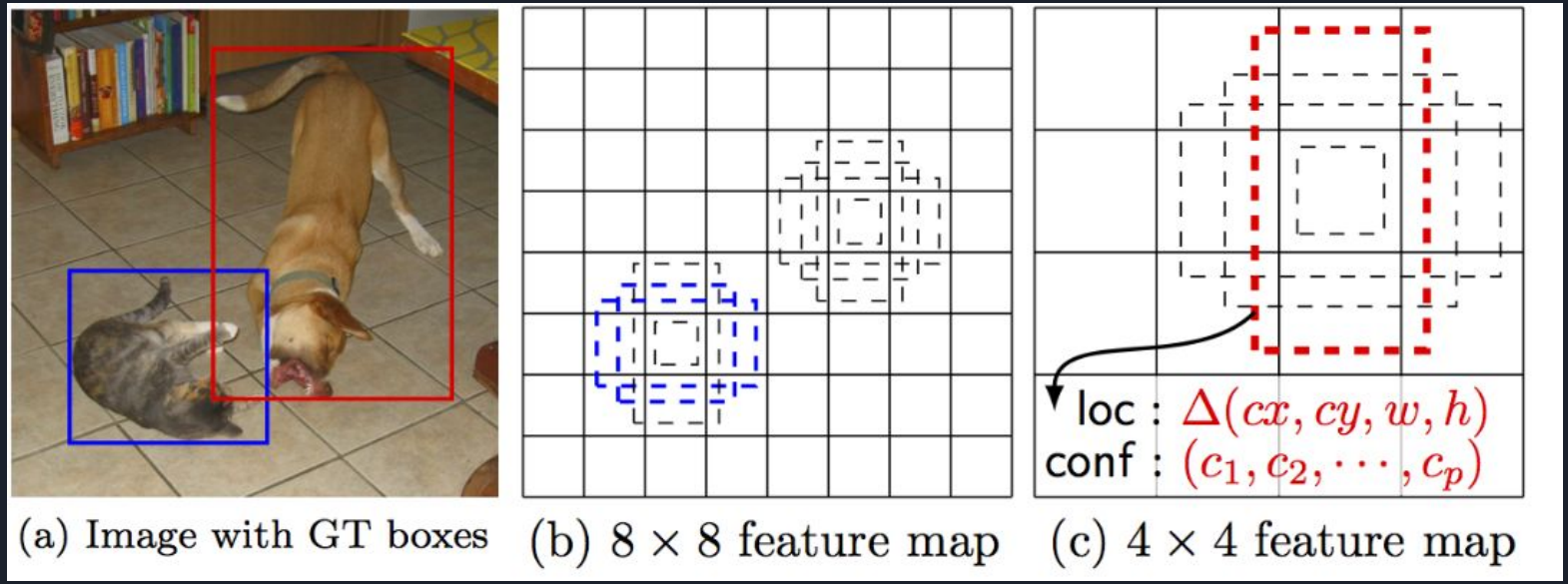
- Na **SSD** as bounding boxes são encontradas ao realizar classificações em **todas as posições da imagem**, utilizando diferentes **formas** e **escalas**.

- Gera maior quantidade de **BBoxes** outras técnicas, sendo que a **maioria** não contém um objeto

Solução para a quantidade de Bounding Boxes (BB)

- Utiliza-se **non-maximum suppression** para **unir** bounding box com **muita sobreposição**
- Quando há muitas BBs **semelhantes**, tanto em forma, dimensão e objeto, é mantida só a com **maior acurácia**

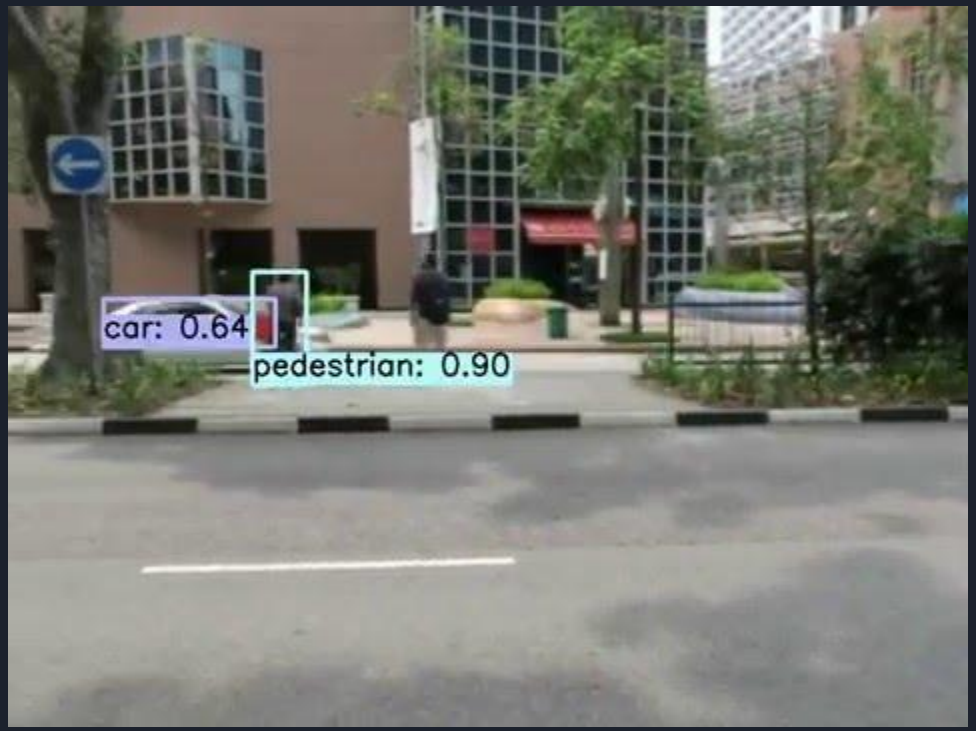
Benefícios da variação de feature maps



SSD - Single Shot Multibox Detector

- Principal contribuição: Faz tudo em “**one-shot**”.
- A principal diferença no processo é que ela **não realiza** a fase de **extração de regiões de interesse**.
- Muito **mais rápida** que as redes **RCNNs**, permitindo aplicação em vídeo em tempo real de **20 a 25 fps**.
- Apresenta uma acurácia competitiva (entre **85** e **90%**).

SSD - Single Shot Multibox Detector



YOLO (You Only Look Once)

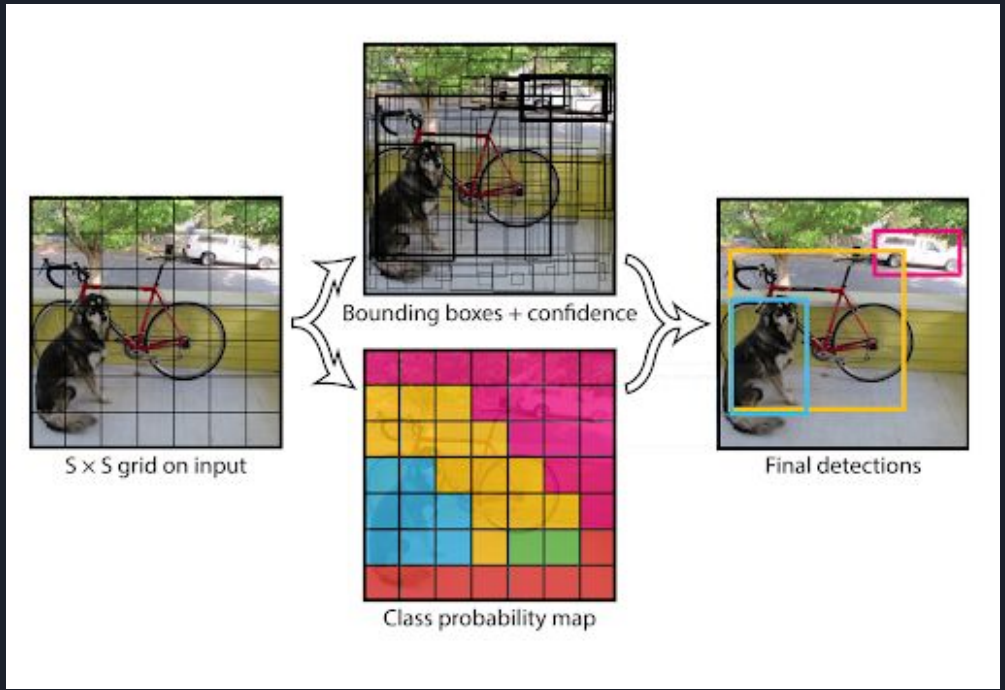
- Bem **similar** ao funcionamento da **SSD**.
- Faz tudo em "**One-Shot**".
- A diferença está na **inferência** das bounding box (utiliza um método próprio).
- Existe a **Yolo**, **Yolo v2**, **YOLO9000** e mais recentemente, a **Yolo v3**
- Roda em vídeos de **60-90 fps** com acurácia de aproximadamente **80%**.

YOLO (You Only Look Once)

- A **confiança** está relacionada à **acurácia do bounding box** e se uma **bounding box contém um objeto**
- $Pr(Classi|Object) * Pr(Object) * IOU = Pr(Classi) * IOU$

YOLO (You Only Look Once)

- Pr(Classi|Object)∗Pr(Object)∗IOU = Pr(Classi)∗IOU



YOLO (You Only Look Once)

- Mais semelhante a uma **Fully convolutional Neural Network**
- A imagem (nxn) é passada pela **parte FCNN** da rede e a saída é uma **predição** (mxm)
- Enxerga todo o problema de Object Detection como um problema de **regressão**
- Possui **menor índice** de **falso positivos** para background

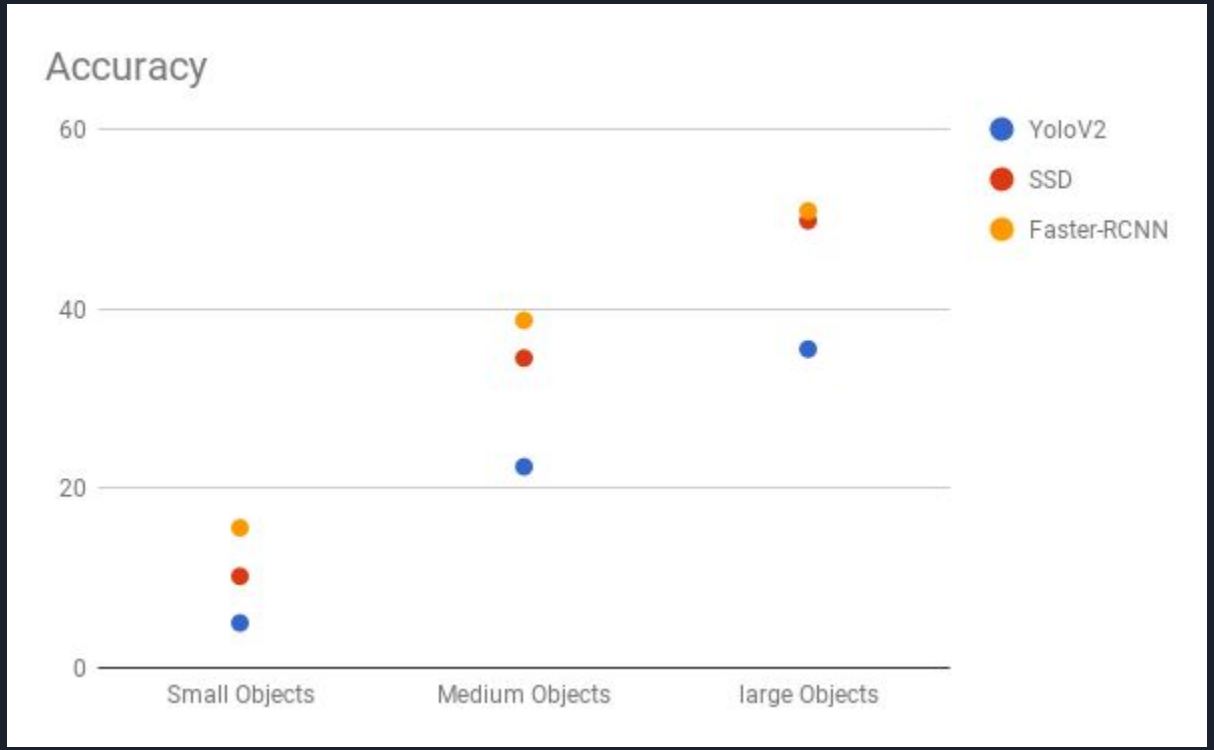
YOLO (You Only Look Once)



Comparação entre Arquiteturas



Comparação entre Arquiteturas



Comparação entre Arquiteturas



Principais datasets

- Os principais datasets utilizados para Detecção de objetos são:
  - **Open Images v4**: 9M imagens com 600 classes
  - **ImageNet**: 450k imagens com 200 classes
  - **COCO**: 120k imagens com 80 classes
  - **Pascal VOC**: 12k imagens com 20 classes
  - **Oxford-IIIT Pet**: 7k imagens com 37 classes
  - **KITTI Vision** : 7k imagens com 3 classes

Tensorflow Object Detection API

Tutorial



Engº Vinícius Araújo Santos

Tutorial: Tensorflow Object Detection API

- Para utilizar os modelos descritos é possível utilizar a API de **Object Detection** do **Tensorflow**

- É um **framework Open Source** baseado em Tensorflow

- Fornece suporte para criação de **modelos** capazes de **localizar** e **identificar objetos** em uma imagem

Tutorial: Tensorflow Object Detection API

Neste tutorial iremos aprender:

1. Como **configurar** o **ambiente** para utilizar este framework
2. Sobre **dados** e **ferramentas de anotação**
3. Organizar os **dados de treinamento**
4. Sobre o **label map**
5. **Configurar** o **treinamento**
6. **Treinar** e **salvar** o modelo

Tutorial: Tensorflow Object Detection API

- Para tal vamos utilizar:

  - **Tensorflow-gpu**
  - **CUDA Toolkit 9.2**
  - **cuDNN 7.1.4 (para CUDA 9.2)**
  - **Anaconda**

Tutorial: Tensorflow Object Detection API

1. Instalar **Anaconda**, **CUDA** e **cuDNN**
   a. Baixe o **CUDA 9.2:**
      - https://developer.nvidia.com/cuda-downloads
      - **37% mais rápido** que **versões anteriores**
      - A **versão 9.2** é de **7 a 12% mais rápida** que a **versão 9.0**
   b. Baixe o **cuDNN 7.1.4**:
      - https://developer.nvidia.com/cudnn
      - Necessária a criação de uma conta
   c. Baixe o **Anaconda**:
      - https://www.anaconda.com/download/

Tutorial: Tensorflow Object Detection API

1. Instalar **Anaconda**, **CUDA** e **cuDNN**
    - A configuração do **cuDNN** inclui a configuração de **variáveis de ambiente**
    - Basta seguir as **instruções de instalação** no site de instalação

Tutorial: Tensorflow Object Detection API

2. Criar o ambiente do **Anaconda** e instalar o **Tensorflow-GPU**:

a. Para **criar o ambiente** basta **executar** em seu **terminal**:
   - > conda create -n ObjectDetection pip python 3.5

b. **Ativar** o ambiente:
   - Windows:
     - > activate ObjectDetection
   - MacOS e Linux:
     - > source activate ObjectDetection

Tutorial: Tensorflow Object Detection API

2. Criar o ambiente do **Anaconda** e instalar o **Tensorflow-GPU**:

b. Para instalar o **Tensorflow-GPU** execute:

- > pip install --ignore-installed --upgrade tensorflow-gpu

c. Instalar bibliotecas necessárias:

- Windows:
  - > conda install -c anaconda protobuf
  - > pip install pillow lxml Cython jupyter matplotlib pandas opencv-python

Tutorial: Tensorflow Object Detection API

2. Criar o ambiente do **Anaconda** e instalar o **Tensorflow-GPU**:

   b. Para instalar o **Tensorflow-GPU** execute:

      - > pip install --ignore-installed --upgrade tensorflow-gpu

   c. Instalar bibliotecas necessárias:

      - Linux:
        - > sudo apt-get install protobuf-compiler python-tk
        - > pip install pillow lxml Cython jupyter matplotlib pandas opencv-python

Tutorial: Tensorflow Object Detection API

3. Baixando o repositório de modelos do tensorflow:

A. **Crie** o diretório **tensorflow-models** em:
   a. C:\ (Windows)
   b. ~/home/ (Linux)

B. Baixe o repositório **Models** do seguinte link:
   a. https://github.com/tensorflow/models
   b. Sugiro **baixá-lo compactado** e colocar a pasta **models-master** dentro do diretório **tensorflow-models**
   c. **Navegue** para o diretório da API de Object Detection:
      i. ~/tensorflow-models/research/object-detection

Tutorial: Tensorflow Object Detection API

3. Baixando o repositório de modelos do tensorflow:

C. Configurando as **variáveis de ambiente**:

1. Execute o seguinte comando no **Windows:**
   a. > set PYTHONPATH=C:\tensorflow-models\models;C:\tensorflow-models\models\research;C:\tensorflow-models\models\research\slim

2. Execute o seguinte comando no **Linux** de dentro da pasta **~/tensorflow-models/models/research/**:
   a. > export PYTHONPATH=$PYTHONPATH:`pwd`:`pwd`/slim

Tutorial: Tensorflow Object Detection API

3. Baixando o repositório de modelos do tensorflow:

D. **Compilando** os **arquivos Protobuf** utilizados pelo Tensorflow para **configurar modelos** e **treinar parâmetros**:

1. Execute o seguinte comando no **Windows** e no **Linux** de dentro da pasta **~/tensorflow-models/models/research/**:
   a. > protoc --python_out=. .\object_detection\protos*.proto
2. Execute o comando no **Windows**:
   a. > python setup.py build
   b. > python setup.py install

Tutorial: Tensorflow Object Detection API

3. Baixando o repositório de modelos do tensorflow:

E. **Teste** a instalação executando o seguinte comando para executar o **Jupyter notebook** object_detection_tutorial.ipynb:

1. Execute o seguinte comando no **Windows** e no **Linux** de dentro da pasta **~/tensorflow-models/models/research/object_detection**:
   a. > jupyter notebook object_detection_tutorial.ipynb

Tutorial: Tensorflow Object Detection API

4. Baixando **modelos já treinados**:

- Já há **vários modelos** prontos no **Tensorflow detection model zoo**:
  - https://github.com/tensorflow/models/blob/master/research/object_detection/g3doc/detection_model_zoo.md
  - **Dentre** os **modelos treinados** encontram-se diferentes modelos da **arquitetura SSD**, **Faster R-CNN** e **RFCN** treinadas nos datasets **COCO**, **KITTI**, **OPEN IMAGES v.3** e **AVA**

Tutorial: Tensorflow Object Detection API

4. Baixando **modelos já treinados**:

Coco

ssd_mobilenet_v1_coco	faster_rcnn_inception_resnet_v2_atrous_coco
ssd_mobilenet_v2_coco	faster_rcnn_inception_resnet_v2_atrous_lowproposals_coco
ssdlite_mobilenet_v2_coco	faster_rcnn_nas
ssd_inception_v2_coco	faster_rcnn_nas_lowproposals_coco
faster_rcnn_inception_v2_coco	mask_rcnn_inception_resnet_v2_atrous_coco
faster_rcnn_resnet50_coco	mask_rcnn_inception_v2_coco
faster_rcnn_resnet50_lowproposals_coco	mask_rcnn_resnet101_atrous_coco
rfcn_resnet101_coco	mask_rcnn_resnet50_atrous_coco
faster_rcnn_resnet101_coco	
faster_rcnn_resnet101_lowproposals_coco	

KITTI

faster_rcnn_resnet101_kitti

AVA

faster_rcnn_resnet101_ava_v2.1

Open Images v.3:

faster_rcnn_inception_resnet_v2_atrous_oid

faster_rcnn_inception_resnet_v2_atrous_lowproposals_oid

Tutorial: Tensorflow Object Detection API

4. Baixando **modelos já treinados**:

- Crie o diretório **trained-models** em:
    - ~/tensorflow-models/research/object-detection
- Basta **copiar** os **modelos baixados** para este diretório e já é possível executar os modelos já treinados
- Para isto o Jupyter Notebook **object_detection_tutorial.ipynb** exemplifica bem os **passos necessários** para realizar a **inferência** em **qualquer imagem**
- A mesma abordagem **pode ser aplicada para vídeos**

Tutorial: Tensorflow Object Detection API

5. Preparação para o **treinamento** de um novo **modelo:**

- Para este passo usaremos o diretório do GitHub:
  - “**How To Train an Object Detection Classifier for Multiple Objects Using TensorFlow (GPU) on Windows 10**”
  - https://github.com/EdjeElectronics/TensorFlow-Object-Detection-API-Tutorial-Train-Multiple-Objects-Windows-10
  - Faça o **download** e extraia seu conteúdo para dentro do diretório:
    - ~/tensorflow-models/research/object-detection

Tutorial: Tensorflow Object Detection API

5. Preparação para o **treinamento** de um novo **modelo:**

- Treinamento para **detecção** de **cartas de baralho**



Tutorial: Tensorflow Object Detection API

5. Organizando os **dados** e a **ferramentas de anotação**

- É possível utilizar um dos **dataset citados anteriormente**
    - COCO, KITTI, Open Images v.3 e v.4, etc.
- Estes datasets **já estão organizados** e o treinamento é simples.
- Neste tutorial faremos o **treinamento** a partir de um **novo dataset não anotado**
- Aqui as imagens devem ter no máximo **200KB** e resoluções menores que **720x1280** para que o treinamento seja rápido

Tutorial: Tensorflow Object Detection API

5. Organizando os **dados** e a **ferramentas de anotação**

- É possível utilizar fotos obtidas através de um **crawler no Google Images**, por exemplo
- Também **fotos que você tenha tirado** com uma **câmera** ou **celular**
- Isto porque utilizaremos uma **ferramenta** para fazer a **anotação**

Tutorial: Tensorflow Object Detection API

5. Organizando os **dados** e a **ferramentas de anotação**

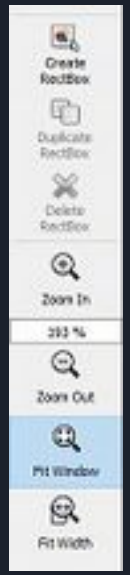
- Para isso utilizaremos a **ferramenta Open Source** chamada **LabelImage**
- Ela está **disponível** no **Github** bem como as **instruções para sua execução** no seguinte link:
    - https://github.com/tzutalin/labelImg

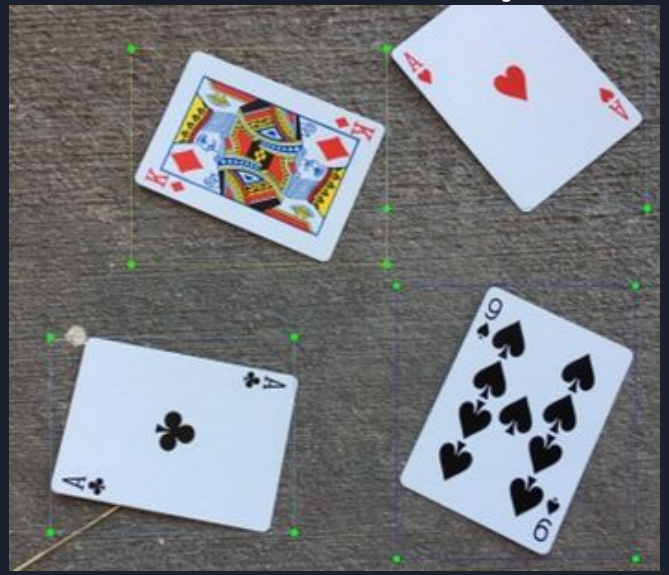
Tutorial: Tensorflow Object Detection API

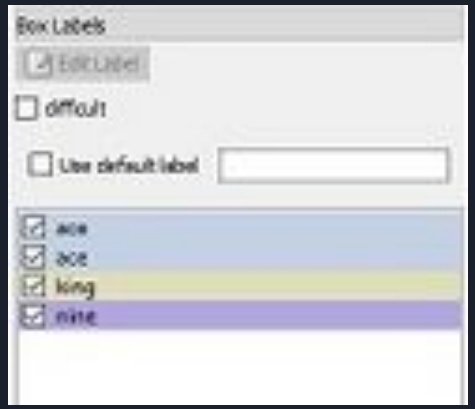
5. Organizando os **dados** e a **ferramentas de anotação**

- Ela é bem direta em sua utilização









Tutorial: Tensorflow Object Detection API

5. Organizando os **dados** e a **ferramentas de anotação**

- **Existem outras ferramentas** e é possível encontrar algumas outras através do link:
  - https://en.wikipedia.org/wiki/List_of_manual_image_annotation_tools
- **Encontre** a que **mais goste**...

Tutorial: Tensorflow Object Detection API

5. Organizando os **dados** e a **ferramentas de anotação**

- Caso queria **treinar em imagens** que obteve **sem anotação**, lembre-se de dividir a base dados em **conjuntos de treinamento** e **teste**.
- Proporções de **20/80 %** ou **30/70%** são sugeridas
- Insira-as no diretório **~/tensorflow-models/models/research/object_detection/images**

Tutorial: Tensorflow Object Detection API

5. **Preparando** os **labels** gerados pelo **LabelImage**:

- Necessário **criar** os **TFRecords** utilizados pela API para o treinamento.
- Para isso serão utilizado os scripts “xml_to_csv.py” e “generate_tfrecord.py ” que foram **modificados** dos originais do “Dat Tran’s Raccoon Detector dataset”.

Tutorial: Tensorflow Object Detection API

5. **Preparando** os **labels** gerados pelo **LabelImage**:

- O **LabelImage** representa os **labels** como arquivos '**.xml**'
- Necessário converter para '**.csv**'
- Do diretório **~/tensorflow-models/models/research/object_detection** execute o seguinte comando:
  - > python xml_to_csv.py
- Os arquivos '**train_labels.csv**' e '**test_labels.csv**' são criados no diretório **~object_detection/images**

Tutorial: Tensorflow Object Detection API

6. Preparando para a criação dos **label maps**:

- Os **labels maps** são mapeamentos feitos do nome da classe definido dentro do **LabelImage** para valores numéricos que permite treinar em **diferentes tipos de anotações**
- Para modificar o **label map** para **imagens** e **classes novas** basta editar o arquivo '**generate_tfrecord.py**'
- Neste arquivo basta **modificar** a **função** existente na **linha 31**, chamada '**class_text_to_int**' como a seguir:

Tutorial: Tensorflow Object Detection API

6. Preparando para a criação dos **label maps**:

- Para o exemplo de **detecção de cartas** o **label map** seria:

```
# TO-DO replace this with label map
def class_text_to_int(row_label):
    if row_label == 'nine':
        return 1
    elif row_label == 'ten':
        return 2
    elif row_label == 'jack':
        return 3
    elif row_label == 'queen':
        return 4
    elif row_label == 'king':
        return 5
    elif row_label == 'ace':
        return 6
    else:
        None
```

Tutorial: Tensorflow Object Detection API

6. Preparando para a criação dos **label maps**:

- Caso queria **detectar outros objetos** o **label map** pode ser **modificado** da seguinte maneira:

```
# TO-DO replace this with label map
def class_text_to_int(row_label):
    if row_label == 'basketball':
        return 1
    elif row_label == 'shirt':
        return 2
    elif row_label == 'shoe':
        return 3
    else:
        None
```

Tutorial: Tensorflow Object Detection API

6. Preparando para a criação dos **label maps**:

- Para criar os **TFRecords** basta executar os comandos:
  - > python generate_tfrecord.py --csv_input=images\train_labels.csv --image_dir=images\train --output_path=train.record
  - > python generate_tfrecord.py --csv_input=images\test_labels.csv --image_dir=images\test --output_path=test.record
- São criados dois arquivos, '**train.record**' and a '**test.record**' em **~/object_detection**

Tutorial: Tensorflow Object Detection API

7. Criando os **label maps**:

- Crie um arquivo em um editor de texto chamado '**labelmap.pbtxt**' no diretório a seguir:
  - **~/tensorflow-models/models/research/object_detection/training**
- Confirme que a extensão é '**.pbtxt**'

Tutorial: Tensorflow Object Detection API

7. Criando os **label maps**:

- Insira o seguinte **label map**:

```
item {
    id: 1
    name: 'nine'
}
item {
    id: 2
    name: 'ten'
}
item {
    id: 3
    name: 'jack'
}
item {
    id: 4
    name: 'queen'
}
item {
    id: 5
    name: 'king'
}
item {
    id: 6
    name: 'ace'
}
```

Tutorial: Tensorflow Object Detection API

7. Criando os **label maps**:

- Modifique o **label map** caso treine em **outras classes**:

```
item {
    id: 1
    name: 'basketball'
}
item {
    id: 2
    name: 'shirt'
}
item {
    id: 3
    name: 'shoe'
}
```

Tutorial: Tensorflow Object Detection API

8. Configurando o **treinamento**:

- Agora o **parâmetros de treinamento** deve ser definidos
- Lembra-se dos **vários modelos** prontos no **Tensorflow detection model zoo**? Eles serão usados como base para o treinamento
- Baixe um dos arquivos para o diretório **~/tensorflow-models/research/object-detection/trained-models**

Tutorial: Tensorflow Object Detection API

8. Configurando o **treinamento**:

- **Descompacte** o arquivo e você deve encontrar algo semelhante a isso.
  - Um **proto** de um grafo de modelo (graph.pbtxt)
  - Um **checkpoint** (model.ckpt.data-00000-of-00001, model.ckpt.index, model.ckpt.meta)
  - Um grafo **congelado** com **pesos** (frozen_inference_graph.pb)
  - Um **arquivo de configuração** usado para **gerar o grafo** (pipeline.config)

Tutorial: Tensorflow Object Detection API

8. Configurando o **treinamento**:

- Para este exemplo utilizarei a arquitetura 'faster_rcnn_inception_v2_pets'
  - Navegue até o diretório **~/tensorflow-models/research/object-detection/samples/configs**
  - Copie o arquivo '**faster_rcnn_inception_v2_pets.config**' para o diretório **~/tensorflow-models/research/object-detection/training**

Tutorial: Tensorflow Object Detection API

8. Configurando o **treinamento**:

- É necessário fazer mudanças no arquivo ‘‘**faster_rcnn_inception_v2_pets.config**’

  - **Linha 9**: coloque o número de classes do problema
    - **num_classes** : 6
  - **Linha 106**: mude o valor de **fine_tune_checkpoint** substituindo o valor **PATH_TO_BE_CONFIGURED** pelo caminho:
    - **fine_tune_checkpoint**: “C:/tensorflow-models/models/research/object_detection/trained-models/faster_rcnn_inception_v2_coco_2018_01_28”

Tutorial: Tensorflow Object Detection API

8. Configurando o **treinamento**:

- Continuando...
  - **Linha 122**: mude o valor de **input_path** substituindo o valor **PATH_TO_BE_CONFIGURED** pelo caminho:
    - **input_path :** “C:/tensorflow-models/models/research/object_detection/train.record”
  - **Linha 124**: mude o valor de **label_map_path** substituindo o valor **PATH_TO_BE_CONFIGURED** pelo caminho:
    - **label_map_path:** “C:/tensorflow-models/models/research/object_detection/training/labelmap.pbtxt”

Tutorial: Tensorflow Object Detection API

8. Configurando o **treinamento**:

- **Linha 128**: mude o valor de **num_examples**:
  - **num_examples:** 6
- **Linha 136**: mude o valor de **input_path** substituindo o valor **PATH_TO_BE_CONFIGURED** pelo caminho:
  - **label_map_path:** “C:/tensorflow-models/models/research/object_detection/test.record”
- **Linha 124**: mude o valor de **label_map_path** substituindo o valor **PATH_TO_BE_CONFIGURED** pelo caminho:
  - **label_map_path:** “C:/tensorflow-models/models/research/object_detection/training/labelmap.pbtxt”

Tutorial: Tensorflow Object Detection API

8. Fazendo o **treinamento**:

- Uma vez feitos estes ajustes é possível **realizar o treinamento**.
- **Execute** o seguinte **comando**:
  - > python train.py --logtostderr --train_dir=training/ --pipeline_config_path=training/faster_rcnn_inception_v2_pets.config
  - **Se tudo correu bem**, o **treinamento** deve **iniciar**.
- Caso queria acompanhar o progresso execute o comando:
  - > tensorboard --logdir=training

Tutorial: Tensorflow Object Detection API

9. Salvando o **modelo**:

- Para **salvar o modelo** após treinamento **execute** o **comando**:
  - > python export_inference_graph.py --input_type image_tensor --pipeline_config_path training/faster_rcnn_inception_v2_pets.config --trained_checkpoint_prefix training/model.ckpt-**XXXX** --output_directory inference_graph
- Substitua o valor **XXXX** deste comando por um **valor numérico** que indica a **contagem de treinamento** no caso de realizar varios treinamentos

Tutorial: Tensorflow Object Detection API

10. **Resultados** utilizando **diferentes modelos**:



Model: Faster-RCNN-Inception-V2
Training Time: 3 hours

Model: MobileNet-SSD-V1
Training Time: 8 hours

OBRIGADO!!!